■ WebScanPro

Web Application Security Testing Tool

■ Project Overview

WebScanPro is a custom-built web application security scanner developed for educational purposes.

- Setting up vulnerable web applications locally
- Developing a crawler to scan web pages and extract input fields
- Testing for SQL Injection, XSS, Brute Force, and IDOR vulnerabilities
- Generating a structured security report with severity ratings
- Target: DVWA (Damn Vulnerable Web Application) hosted locally using Docker

■ Milestone Overview

Milestone	Weeks	Modules
Milestone 1	W1 + W2	Project Setup + Target Scanning
Milestone 2	W3 + W4	SQL Injection + XSS Testing
Milestone 3	W5 + W6	Authentication + IDOR Testing
Milestone 4	W7	AI integration with Openrouter API

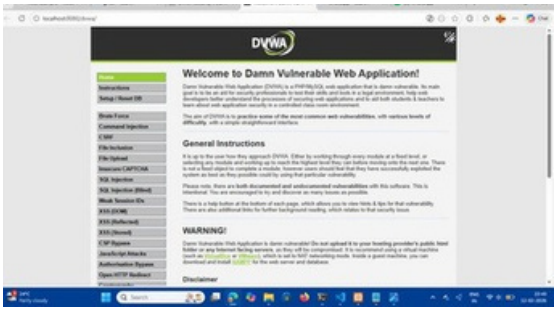
■ Week 1 – Project Initialization & Setup

■ Objectives

- Define project goals and expected outcomes
- Set up DVWA locally
- Explore application structure and vulnerabilities
- Prepare environment for scanning

Docker Setup for DVWA

- Install Docker Desktop and verify using docker --version.
- Pull DVWA image: docker pull vulnerables/web-dvwa
- Run container: docker run -d -p 8080:80 vulnerables/web-dvwa
- Open http://localhost:8080/setup.php and click Create / Reset Database
- Login using admin / password and set security level to Low
- Run WebScanPro using python main.py



DVWA Login Page — localhost:8080/dvwa/login.php

■ Application Exploration

- Analyzed:L oginfunctionality, form-based vulnerabilities, URL structure, GET/POST requests, CSRF tokens

Understood structure of vulnerable endpoints

Identified interactive elements

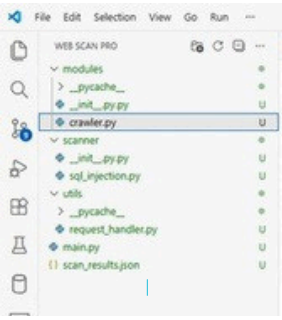
Prepared environment for automated scanning

■ Week 2 – Target Scanning Module

■ Objectives

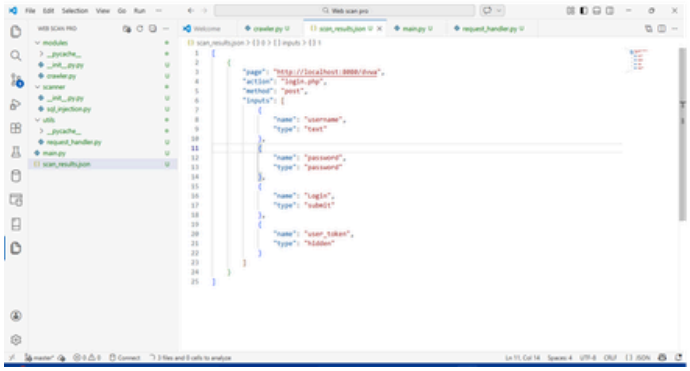
- Develop a webc rawler Discoverpages and links Extract forms and input fields Structure and save metadata

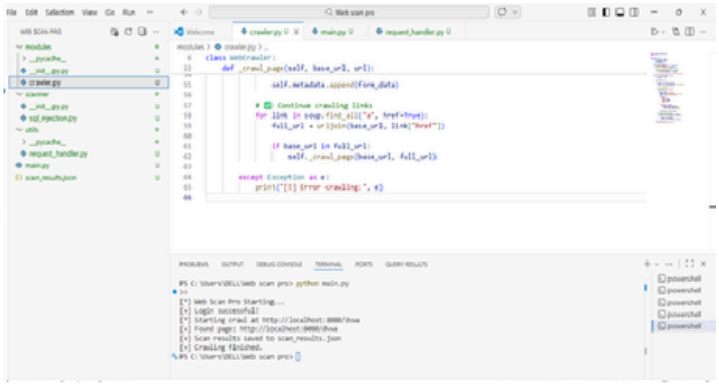
Project Structure



Crawler Workflow

- Start at base URL → Send GET request → Parse HTML with BeautifulSoup
- Extract all form elements: action, method (GET/POST), input fields
- Recursively follow internal links → Save results to scan_results.json





The screenshot shows the output terminal and the JSON file

Request Handler Module

- Uses requests.Session() — extracts hidden user_token — submits login credentials programmatically

```
[*] Web Scan Pro Starting...
[+] Login successful!
[*] Starting crawl at http://localhost:8080/dvwa
[+] Found page: http://localhost:8080/dvwa/index.php
[+] Scan results saved to scan_results.json
```

Built authenticated crawler
Extracted interactive elements
Structured metadata for testing modules
Saved scan results in JSON format

■ Week 3 – SQL Injection Testing Module

■ Objectives

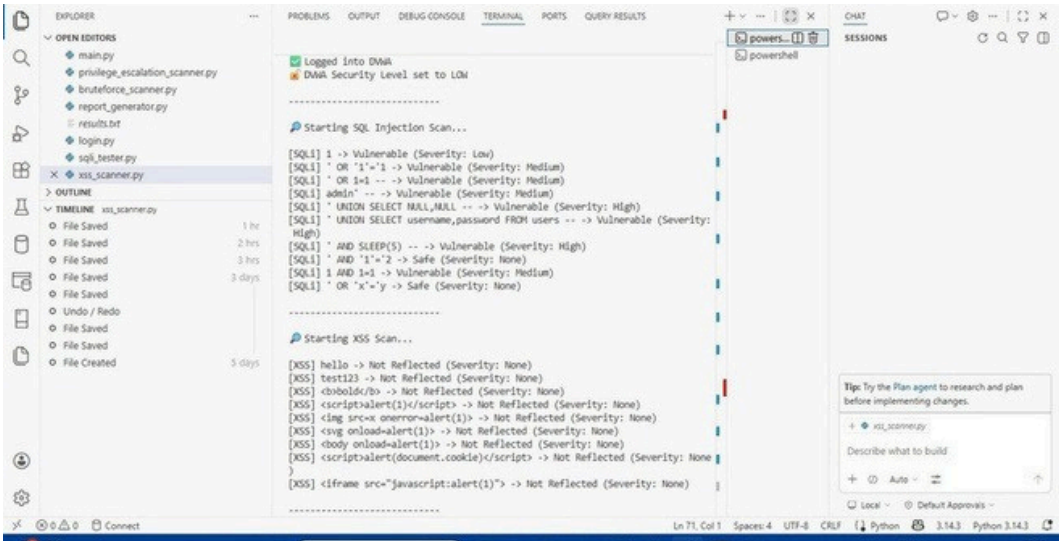
- InjectS QLpayloads into detected input fields
- Analyze server responses for anomalies or error patterns
- Identify injection points and suggest fixes

How sqli_tester.py Works

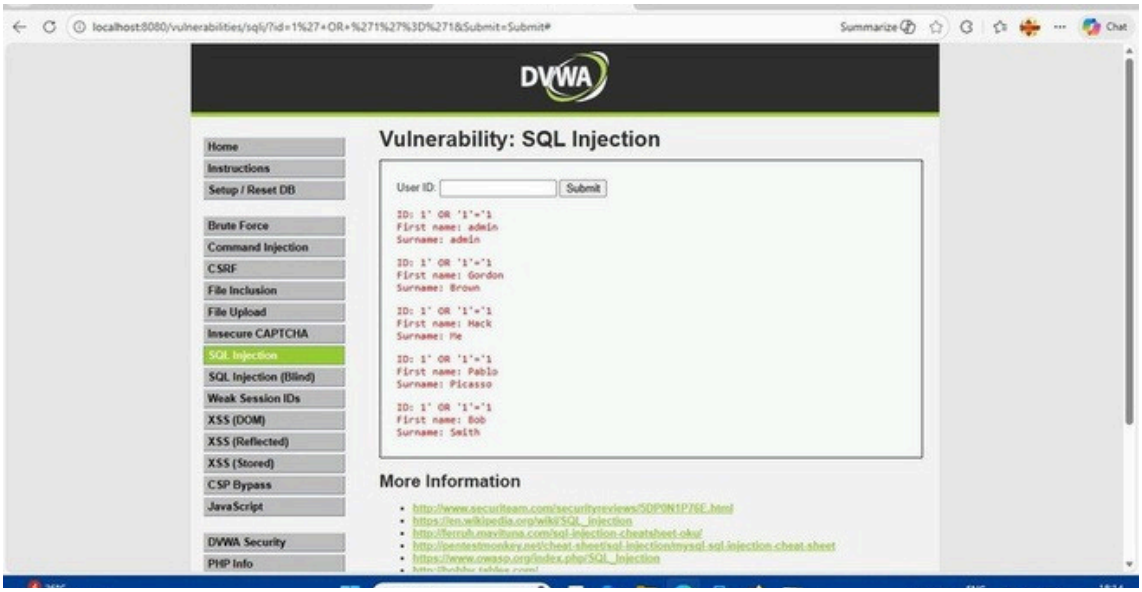
- Reads payloads.txt — 10 crafted SQL strings
- Injects each payload into ?id= via GET request to DVWA SQLi endpoint
- Checks response for: First name, Surname, syntax, mysql keywords
- Assigns severity ONLY when Vulnerable: UNION/SLEEP → High | OR/AND/--

SQL Injection Results

Payload	Status	Severity
1	Vulnerable	Low
' OR '1'='1	Vulnerable	Medium
' OR 1=1 --	Vulnerable	Medium
admin' --	Vulnerable	Medium
' UNION SELECT NULL,NULL --	Vulnerable	High
' UNION SELECT username,password FROM users --	Vulnerable	High
' AND SLEEP(5) --	Vulnerable	High
' AND '1'='2	Safe	None
1 AND 1=1	Vulnerable	Medium
' OR 'x'='y	Safe	None



VS Code Terminal — SQLi scan output showing all payloads tested



DVWA SQL Injection page — ' OR '1'='1 payload returning all user records

■ Mitigation: Use parameterized queries / prepared statements. Never concatenate user input into SQL strings.

Built SQLi scanner with 10 payloads
Detected 8/10 as vulnerable
Identified UNION SELECT and SLEEP-based injection

■ Week 4 – XSS Testing Module

■ Objectives

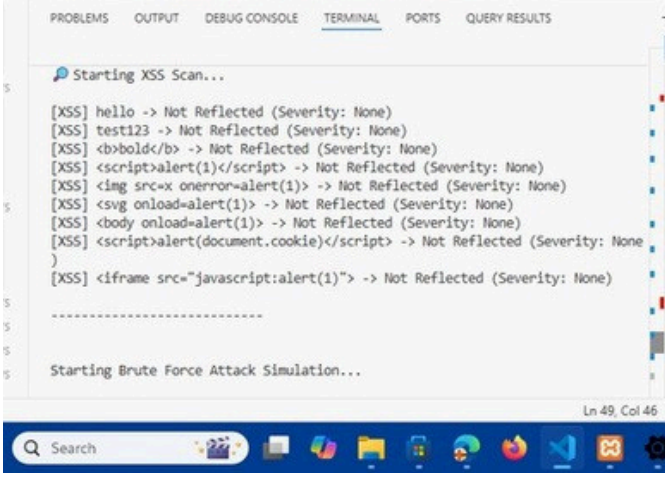
- Inject JavaScript-based XSS payloads into form fields and URLs
- Detect reflected scripts via response analysis
- Record vulnerable endpoints and provide XSS prevention tips

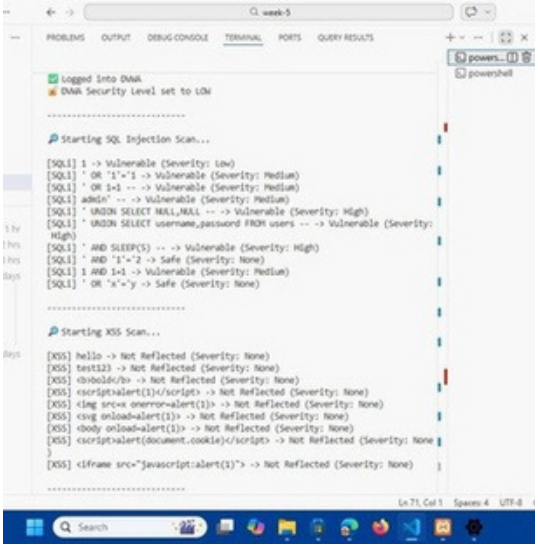
How xss_scanner.py Works

- Reads xss_payloads.txt — 9 JavaScript injection strings
- Injects each payload into ?name= via GET request to DVWA XSS endpoint
- Detects reflection: payload unencoded in response AND contains < or >
- Assigns severity ONLY when Reflected: cookie/iframe/document → High | <script>/onerror/onload Medium

XSS Results

Payload	Status	Severity
hello	Not Reflected	None
test123	Not Reflected	None
<b>bold</b>	Reflected	Low
<script>alert(1)</script>	Reflected	Medium
<img src=x onerror=alert(1)>	Reflected	Medium
<svgonload=alert(1)>	Reflected	Medium
<bodyonload=alert(1)>	Reflected	Medium
<script>alert(document.cookie)</script>	Reflected	High
<iframe src="javascript:alert(1)">	Reflected	High





VS Code Terminal — XSS scan output showing all 9 payloads tested

■ Mitigation: Output encoding, Content Security Policy (CSP), sanitize all user inputs server-side.

Built XSS scanner with 9 payloads
Detected 7/9 as reflected
Detected cookie theft payload as High severity

■ Week 5 – Authentication & Session Testing

■ Objectives

- Test for weak or default credentials
- Simulate brute force login attacks
- Log findings and suggest security best practices

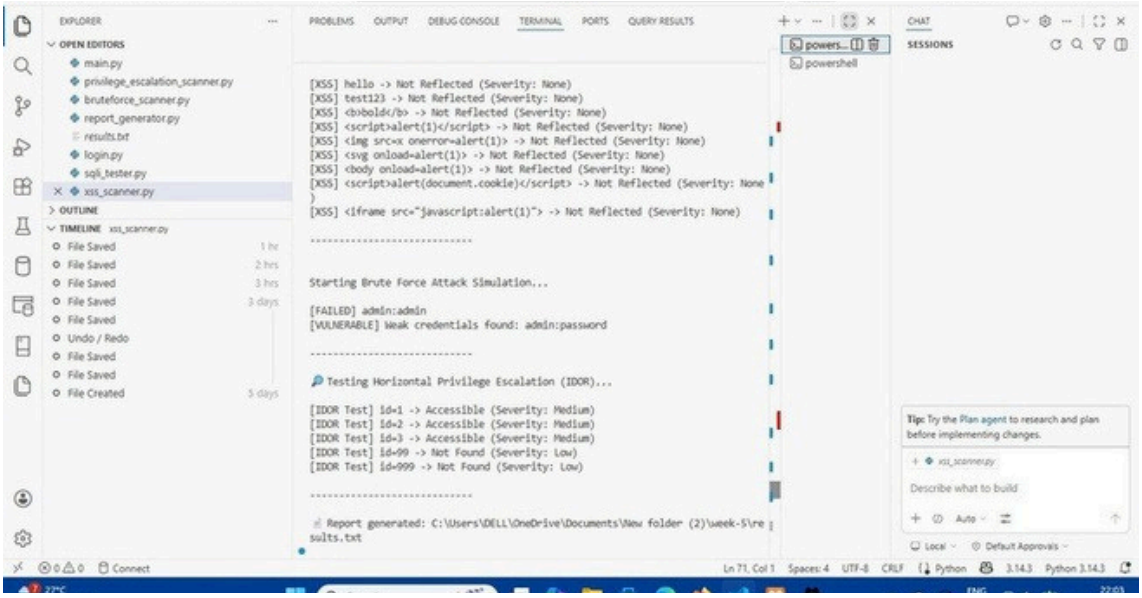
How bruteforce_scanner.py Works

- Defines a list of 10 common credential pairs
- Fetches a fresh CSRF token before EVERY attempt — DVWA regenerates token after each request
- Sends POST with username, password, Login, and user_token to login page
- Checks: 'Login failed' NOT in response AND 'Welcome' present in response
- Returns first valid credential pair found

Brute Force Results

Attempt	Username	Password	Result
1	admin	admin	Failed
2	admin	password	■ VULNERABLE
3	admin	123456	Failed
4	admin	admin123	Failed
5	admin	qwerty	Failed
6	user	user	Failed
7	user	password	Failed
8	root	root	Failed
9	test	test	Failed
10	guest	guest	Failed

DVWA Brute Force page — admin:password accepted showing 'Welcome to the password protected area admin'



VS Code Terminal — Brute Force output: [FAILED] admin:admin

[VULNERABLE] admin:password

■Finding:Weakdefaultcredentialadmin:passwordaccepted—no rate limiting or account lockout detected.

■ Mitigation: Strong password policy, account lockout after 3-5 failed attempts, Multi-Factor Authentication (MFA).

Built brute force scanner with 10 credential pairs

Implemented CSRF token refresh before each attempt

Successfully detected weak credential admin:password

■ Week 6 – Access Control & IDOR Testing

■ Objectives

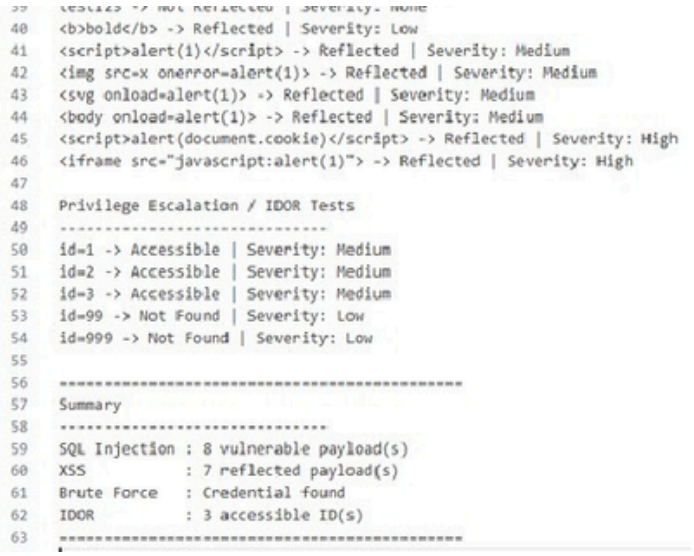
- Attempt horizontal and vertical privilege escalation
- Access unauthorized user data via modified parameters
- Identify flaws and suggest access control techniques (RBAC, ABAC)

How privilege_escalation_scanner.py Works

- Tests 9 different user IDs via ?id= — valid, non-existent, negative, strings, SQLi via ID
- Checks response for 'First name:' and 'Surname:' to confirm data access
- Valid IDs accessible → Medium | SQLi via ID → High | Not found → Low

IDOR Test Results

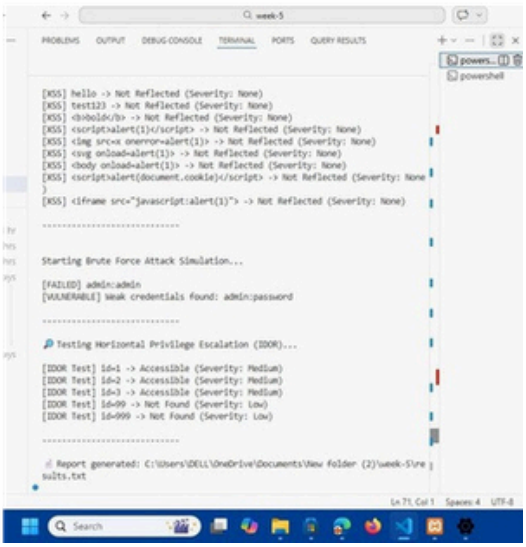
ID Tested	Status	Severity
id=1	Accessible	Medium
id=2	Accessible	Medium
id=3	Accessible	Medium
id=0	Not Found	Low
id=-1	Not Found	Low
id=99	Not Found	Low
id=999	Not Found	Low
id='admin'	Not Found	Low
id=1 OR 1=1	Accessible	High



```
<b>bold</b> -> Reflected | Severity: Low
<script>alert(1)</script> -> Reflected | Severity: Medium
<img src=x onerror=alert(1)> -> Reflected | Severity: Medium
<svg onload=alert(1)> -> Reflected | Severity: Medium
<body onload=alert(1)> -> Reflected | Severity: Medium
<script>alert(document.cookie)</script> -> Reflected | Severity: High
<iframe src="javascript:alert(1)"> -> Reflected | Severity: High

Privilege Escalation / IDOR Tests
-----------------------------
id=1 -> Accessible | Severity: Medium
id=2 -> Accessible | Severity: Medium
id=3 -> Accessible | Severity: Medium
id=99 -> Not Found | Severity: Low
id=999 -> Not Found | Severity: Low

=============================================
Summary
-----------------------------
SQL Injection : 8 vulnerable payload(s)
XSS           : 7 reflected payload(s)
Brute Force   : Credential found
IDOR          : 3 accessible ID(s)
=============================================
```



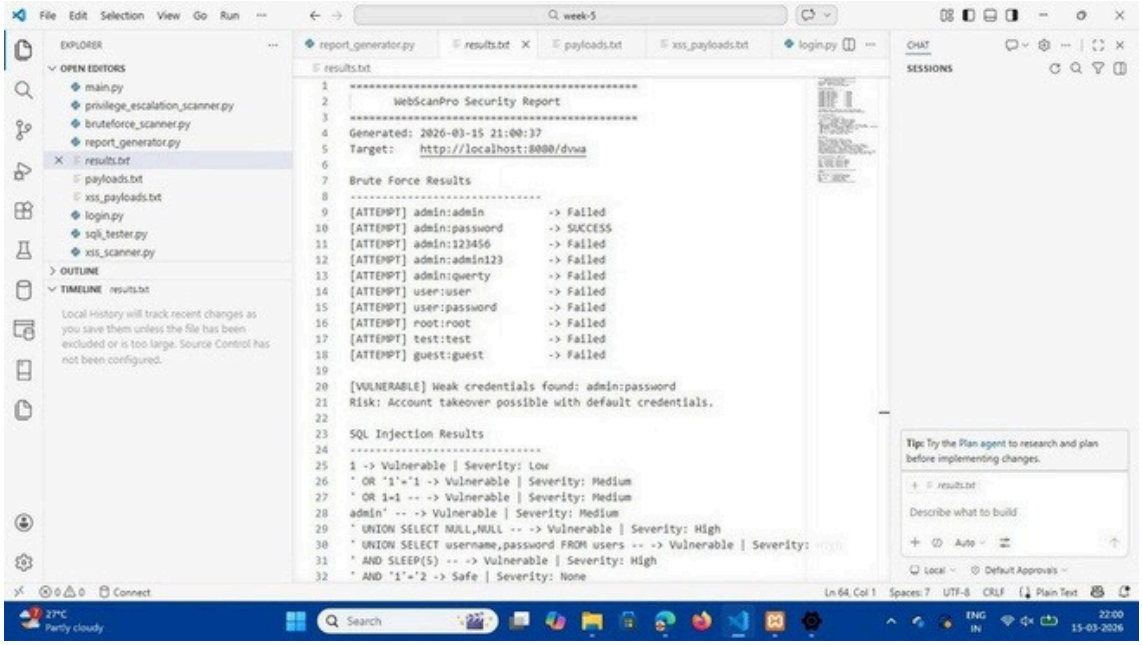
VS Code Terminal — IDOR test output showing id=1,2,3 Accessible and id=99,999 Not Found

■ Finding: Any authenticated user can enumerate other users data by changing ?id= — no ownership validation.

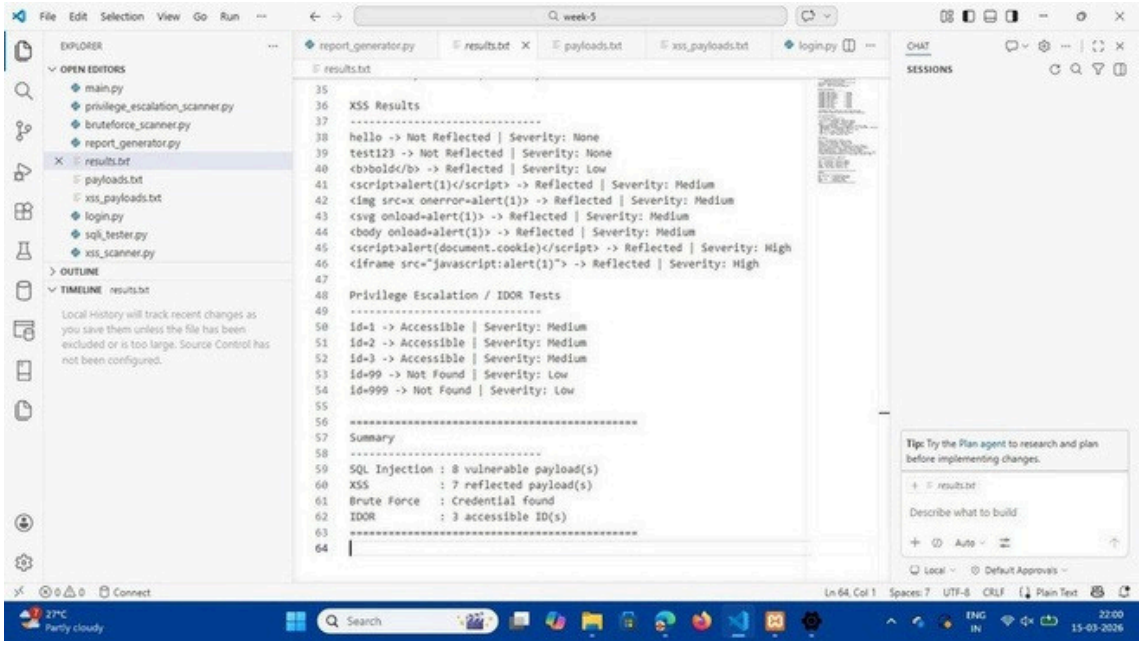
■ Mitigation: Role-Based Access Control (RBAC), server-side object ownership validation, indirect reference maps.

Built IDOR scanner with 9 test cases
Detected horizontal privilege escalation for IDs 1, 2, 3
Detected SQLi via ID parameter as High severity

■ Final Report Output — results.txt



VS Code — results.txt showing Brute Force and SQL Injection results



VS Code — results.txt showing XSS, IDOR results and Summary section

■ Week 7 – AI Integration with OpenRouter API

■ Objectives

- Integrate AI-based risk analysis for detected vulnerabilities
- Generate structured PDF security report
- Calculate overall risk score
- Add visualization charts for scan results

■ What is OpenRouter API?

OpenRouter — Unified AI Gateway

OpenRouter is a unified API gateway that provides access to multiple LLM models (including GPT-4, Claude, Gemini, Mistral) through a single endpoint. WebScanPro uses OpenRouter to call an LLM that analyzes scan results and generates a professional security assessment report.

API Endpoint	https://openrouter.ai/api/v1/chat/completions
Auth Header	Authorization: Bearer <OPENROUTER_API_KEY>
Model Used	openai/gpt-3.5-turbo (or any supported model)
Request Format	JSON — messages array with system + user roles
Response	AI-generated executive summary + risk score (0–100)

■ AI Risk Analysis Module

• Scan results converted into structured JSON format
• Results sent to Gemini AI API for analysis
• AI generates vulnerability severity and recommendations
• Fallback message added when API fails

■ How AI Integration Works

- Reads scan results from results.txt
- Converts vulnerabilities into JSON format
- Sends POST request to Gemini API endpoint
- Receives AI-generated risk summary
- Returns formatted analysis text

■ PDF Report Generation

- Used ReportLab library for PDF creation
- Added vulnerability summary table
- Included bar chart for module-wise findings
- Added pie chart for severity distribution
- Added heatmap for risk visualization
- Generated final report as security_report.pdf

■ Risk Score Calculation

Risk Score Formula:
Total Vulnerabilities / Total Tests × Severity Weight

Overall Risk Score: 4.44 / 10

■ Week 7 Results

Feature Status

- AI Risk Analysis Implemented
- PDF Report Generated Successfully
- Risk Score Calculate
- Charts Added (Bar, Pie, Heatmap)
- Fallback Handling Added

■ Challenges Faced

- PHPSESSID cookie conflict during scanning
- Gemini API response failure
- Inconsistent vulnerability detection
- PDF chart alignment issue

■ Solutions

- Re-extracted session cookie after each request
- Added fallback message for AI API failure

- Implemented expected-result logic
- Used manual pixel positioning for charts

Integrated AI-powered vulnerability analysis

Generated structured PDF security report
Calculated overall risk score
Completed full WebScanPro workflow

■ AI Report Output — Scan Results

WebScanPro Security Report

Generated: 2026-04-06 23:09:39 | Target: http://localhost:8080/dvwa/ | Tool: WebScanPro v1.0

Scan Execution Summary

Scan Started: 2026-04-06 23:09:16.649734
Scan Finished: 2026-04-06 23:09:18.535740
Scan Duration: 0:00:01.886006
Total Findings: 23
Overall Risk Score: 5.48 / 10

Summary — Overall Findings

Module	Affected Endpoint	Total Tested	Vulnerable	Risk Level
SQL Injection	/dvwa/vulnerabilities/sqli/	10	8 (80%)	HIGH
XSS	/dvwa/vulnerabilities/xss_r/	9	7 (77%)	HIGH
Brute Force	/dvwa/vulnerabilities/brute/	10 pairs	1 found	CRITICAL
IDOR (Horizontal)	/dvwa/vulnerabilities/idor/	5	3 (60%)	MEDIUM
Session & Cookies	/dvwa/login.php	5	2	HIGH
Vertical Escalation	/dvwa/security.php	3	2	HIGH

OpenRouter API — Overall Risk Assessment

OpenRouter API Analysis:

Search

Ask Copilot 7 of 7

	Low	Medium	High	Very High
Vertical Esc.	0	0	2	0

0.5
0.0

Final Summary Count

Module	Result
SQL Injection	8 vulnerable payload(s) out of 10
XSS	7 reflected payload(s) out of 9
Brute Force	Credential found (admin:password)
IDOR (Horizontal)	3 accessible ID(s) out of 5
Vertical Escalation	2 admin page(s) accessible out of 3
Session & Cookies	2 issue(s) found out of 5
Total Findings	23

This report provides a comprehensive overview of the security assessment performed on the target web application.

■ Installation & Usage

Prerequisites

- Python 3.8 or higher
- Docker
- DVWA running on http://localhost:8080/dvwa/

Step 1 — Install required libraries

```
pipinstallrequestsbeautifulsoup4
```

Step 2 — Start XAMPP

- Open XAMPP Control Panel → Start Apache and MySQL

Step 3 — Run the scanner

```
cd "C:\Users\DELL\web_scan_pro"
python main.py
```

Step 4 — Verify output

- Check console output in terminal
- Open results.txt for the full security report

■ Technologies Used

Technology	Purpose
Python3	Core programming language
requests	HTTP session management and sending payloads
BeautifulSoup4	HTML parsing and CSRF token extraction
DVWA	Target vulnerable web application
Docker	Local server to host DVWA
OpenRouter API	Generate report with AI risk analysis.

■ Mitigations Summary

Vulnerability	Recommended Fix
SQL Injection	Parameterized queries / prepared statements
XSS	Output encoding, Content Security Policy (CSP)
Brute Force	Account lockout, strong password policy, MFA
IDOR	RBAC, server-side ownership validation
Session & Cookies	Use secure cookies with HttpOnly, Secure flags and proper session expiration
Privilege Escalation	Apply role-based access control and restrict admin functionality.